malloc

Introduction

Merci de respecter les règles suivantes :

- Restez poli, courtois, respectueux et constructif pendant tout le processus de correction.
Le bien-être de la communauté en dépend.

- Identifiez avec la personne (ou le groupe) évaluée les éventuels dysfonctionnements du travail.
Prenez le temps de discuter et de débattre des problèmes que vous avez identifiés.

- Vous devez tenir compte du fait que vos pairs ont pu comprendre différemment les consignes du projet et l'étendue de ses fonctionnalités.

- Gardez toujours l'esprit ouvert et notez aussi honnêtement que possible. La pédagogie n'est valable que si l'évaluation par les pairs est menée sérieusement.

Consignes

Vous DEVEZ lancer les tests demandés.

Attention : ce projet est assez complexe, le résultat et l'implémentation sont subjectifs. Vous devez garder à l'esprit l'objectif : « Ce projet consiste à implémenter un mécanisme d'allocation dynamique de la mémoire. »

Pièces jointes

subject.pdf

test0.c (lien non disponible)

test1.c (lien non disponible)

test2.c (lien non disponible)

test3.c (lien non disponible)

test4.c (lien non disponible)

test5.c (lien non disponible)

test6.c (lien non disponible)

run_linux.sh (lien non disponible)

run_mac.sh (lien non disponible)

Tests préliminaires

Vérifiez d'abord les éléments suivants

Instructions préliminaires

Vérifiez d'abord les éléments suivants :

- Il y a quelque chose dans le dépôt git
- Le fichier author est valide
- Un Makefile est présent et contient toutes les règles demandées
- Aucune erreur de norme, la Norminette fait foi
- Pas de triche (fonctions non autorisées...)
- 2 globales sont autorisées : une pour gérer les allocations, et une pour gérer les allocations thread-safe

Si un élément de cette liste n'est pas respecté, la notation s'arrête. Utilisez le flag approprié.
Vous pouvez continuer à débattre du projet, mais la notation ne sera pas appliquée.

◯
Oui
◉
Non

Compilation de la bibliothèque

Nous allons d'abord vérifier que la compilation de la bibliothèque génère bien les fichiers demandés, en modifiant HOSTTYPE :

```
$> export HOSTTYPE=Testing
$> make re ...
$> ln -s libft_malloc_Testing.so libft_malloc.so
$> ls -l libft_malloc.so
libft_malloc.so -> libft_malloc_Testing.so
$>
```

Le Makefile utilise-t-il bien HOSTTYPE pour définir le nom de la bibliothèque (libft_malloc_$HOSTTYPE.so) et crée-t-il un lien symbolique libft_malloc.so pointant vers libft_malloc_$HOSTTYPE.so ?

Si ce n'est pas le cas, la soutenance s'arrête.

◯
Oui
◉
Non

Export des fonctions

Vérifiez avec nm que la bibliothèque exporte bien les fonctions malloc, free, realloc et show_alloc_mem.

```
$> nm libft_malloc.so
0000000000000000 T _free
0000000000000000 T _malloc
0000000000000000 T _realloc
0000000000000000 T_show_alloc_mem
U _mmap
U _munmap
U _getpagesize
U _write
U dyld_stub_binder
$>
```

Les fonctions exportées par la bibliothèque sont marquées T, les fonctions non définies utilisées depuis une autre bibliothèque sont marquées U (l'ordre des lignes peut varier).

Si les fonctions ne sont pas exportées, la soutenance s'arrête.

◯
Oui
◉
Non

Tests des fonctionnalités

Commencez par créer un script qui modifiera uniquement les variables d'environnement pendant l'exécution d'un programme de test. Il s'appellera run.sh et sera exécutable :
POUR MAC :
$> cat run.sh
#!/bin/sh
export DYLD_LIBRARY_PATH=.
export DYLD_INSERT_LIBRARIES="libft_malloc.so"
export DYLD_FORCE_FLAT_NAMESPACE=1
$@
$>
POUR LINUX :
$> cat run.sh
#!/bin/sh
export LD_LIBRARY_PATH=.
export LD_PRELOAD=libft_malloc.so
$@
$>

Mise en place du script

Créez un script nommé `run.sh` et rendez-le exécutable :

POUR MAC :

```
$> cat run.sh
#!/bin/sh
export DYLD_LIBRARY_PATH=.
export DYLD_INSERT_LIBRARIES="libft_malloc.so"
export DYLD_FORCE_FLAT_NAMESPACE=1
$@
$>
```

POUR LINUX :

```
$> cat run.sh
#!/bin/sh
export LD_LIBRARY_PATH=.
export LD_PRELOAD=libft_malloc.so
$@
$>
```

◯
Oui
◉
Non

Test de malloc - Référence

Nous allons d'abord écrire un premier programme de test qui n'utilise pas malloc, afin d'avoir une base de comparaison.

Utilisez le fichier test0.c joint au barème

ATTENTION : si vous utilisez une VM Linux, assurez-vous d'utiliser le binaire time installable avec apt (sudo apt install time), sinon vous n'aurez pas accès à l'option -v

```
MAC: $> gcc -o test0 test0.c
$> /usr/bin/time -l ./test0
0.00

LINUX: $> gcc -o test0 test0.c
$> /usr/bin/time -v ./test0
0.00
Command being timed: "./test0"
User time (seconds): 0.00
System time (seconds): 0.00
Percent of CPU this job got: 0%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
Average shared text size (kilobytes): 0
Average unshared data size (kilobytes): 0
Average stack size (kilobytes): 0
Average total size (kilobytes): 0
Maximum resident set size (kilobytes): 412
Average resident set size (kilobytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming frame) page faults: 130
Voluntary context switches: 0
Involuntary context switches: 0
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
$>
```

Nous allons ensuite ajouter malloc et écrire dans chaque allocation pour nous assurer que la page mémoire est bien allouée en mémoire physique par la MMU. Le système n'alloue réellement la page que si l'on écrit dedans : même si l'on fait un mmap plus grand que la demande de malloc, cela ne modifiera pas les « page reclaims ».

◯
Oui
◉
Non

Test de malloc - Allocation mémoire

À l'aide du fichier test1.c fourni en pièce jointe, lancez les tests suivants :

```
$> gcc -o test1 test1.c
$> /usr/bin/time -l ./test1
(Utiliser -v sous Linux)
Command being timed: "./test1"
User time (seconds): 0.00
System time (seconds): 0.00
Percent of CPU this job got: 0%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.00
Average shared text size (kilobytes): 0
Average unshared data size (kilobytes): 0
Average stack size (kilobytes): 0
Average total size (kilobytes): 0
Maximum resident set size (kilobytes): 1500
Average resident set size (kilobytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming frame) page faults: 402
Voluntary context switches: 0
Involuntary context switches: 0
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Notre programme test1 a demandé 1024 fois 1024 octets, soit 1 Mo. On peut donc vérifier en faisant la différence avec le programme test0 :
- soit entre les lignes « maximum resident set size » : on obtient un peu plus de 1 Mo
- soit entre les lignes « page reclaims », à multiplier par la valeur de getpagesize(3)

Testons maintenant les deux programmes avec notre bibliothèque :

```
$>./run.sh /usr/bin/time -l ./test0
(Utiliser -v sous Linux)
...
Maximum resident set size (kilobytes): 412
Minor (reclaiming frame) page faults: 130
...

$>./run.sh /usr/bin/time -l ./test1
(Utiliser -v sous Linux)
...
Maximum resident set size (kilobytes): 1984
Minor (reclaiming frame) page faults: 525
$>
```

Comptez le nombre de pages utilisées et ajustez la note ainsi :

- moins de 255 pages, la mémoire allouée est insuffisante : 0
- 1023 pages et plus, malloc fonctionne mais consomme au moins 1 page par allocation : 1
- entre 513 et 1022 pages, malloc fonctionne mais le surcoût est trop important : 2
- entre 313 et 512 pages, malloc fonctionne mais le surcoût est très important : 3
- entre 273 et 312 pages, malloc fonctionne mais le surcoût est important : 4
- entre 255 et 272 pages, malloc fonctionne et le surcoût est correct : 5

La personne évaluée peut justifier une autre méthode de comptage des pages allouées (avec un debug existant par exemple).

Notez de 0 (échec) à 5 (excellent)
-- Choisir --

Zones pré-allouées

Vérifiez dans le code source que les zones pré-allouées pour les différentes tailles de malloc permettent de stocker au moins 100 fois la taille maximale de ce type de zone. Vérifiez aussi que la taille des zones est un multiple de getpagesize().

Si l'un de ces points manque, cliquez sur NON.

◯
Oui
◉
Non

Tests de free

Nous allons simplement ajouter un free à notre programme de test :

```
$> cat test2.c
$> gcc -o test2 test2.c

$>./run.sh /usr/bin/time -l ./test2
(Utiliser -v sous Linux)
```

Nous allons comparer le nombre de « page reclaims » à ceux de test0 et test1. S'il y en a autant ou plus que pour test1, le free ne fonctionne pas.

Le free fonctionne-t-il ? (moins de « page reclaims » que test1)

◯
Oui
◉
Non

Qualité de la fonction free

test2 a-t-il au plus 3 « page reclaims » de plus que test0 ?

◯
Oui
◉
Non

Test de realloc

À l'aide du fichier test3.c fourni en pièce jointe, testez ce qui suit :

```
$> gcc -o test3 test3.c
$> ./run.sh ./test3 Bonjour Bonjour
$>
```

Cela fonctionne-t-il comme prévu ?

◯
Oui
◉
Non

Test de realloc++

test4.c est une petite modification de test3.c. Lancez-le : fonctionne-t-il toujours ?

◯
Oui
◉
Non

Gestion des erreurs

À l'aide de test5.c, testez tous les cas particuliers et les erreurs :

```
$> gcc -o test5 test5.c
$> ./run.sh ./test5 Bonjour
```

En cas d'erreur, realloc doit renvoyer NULL. « Bonjour » s'affiche-t-il comme prévu ? Si le programme réagit mal (segfault ou autre), la soutenance s'arrête et vous devez sélectionner le flag « crash ».

◯
Oui
◉

Non

Test de show_alloc_mem

```
$> gcc -o test6 test6.c -L. -lft_malloc
$> ./run.sh ./test6
```

L'affichage correspond-il au sujet et à la répartition TINY/SMALL/LARGE des allocations du projet ?

◯
Oui
◉
Non

Bonus

Les bonus ne seront pris en compte que si TOUS les points précédents sont validés.

Accès concurrents

Le projet gère les accès concurrents des threads à l'aide de mutex et de la bibliothèque pthread.

Comptez les cas applicables :

- Un mutex empêche plusieurs threads d'entrer simultanément dans la fonction malloc
- Un mutex empêche plusieurs threads d'entrer simultanément dans la fonction free
- Un mutex empêche plusieurs threads d'entrer simultanément dans la fonction realloc
- Un mutex empêche plusieurs threads d'entrer simultanément dans la fonction show_alloc_mem

Notez de 0 (échec) à 5 (excellent)
-- Choisir -- ˅

Bonus supplémentaires

S'il y a d'autres bonus, notez-les ici. Les bonus doivent être fonctionnels à 100 % et au minimum utiles. (à l'appréciation du correcteur)

Exemples de bonus :

- Lors d'un free, le projet « défragmente » la mémoire libre en regroupant les blocs libres contigus.
- Malloc dispose de variables d'environnement de débogage.
- Une fonction permet de faire un dump hexadécimal des zones allouées.

- Une fonction permet d'afficher un historique des allocations mémoire effectuées.
- S'il y a d'autres bonus, comptez-les ici. Les bonus doivent être fonctionnels à 100 % et au minimum utiles (à l'appréciation du correcteur).

Notez de 0 (échec) à 5 (excellent)

-- Choisir --